

Maple: A Marketplace of Solana dApps Powered by Agentic Interfaces and RAG Skills

Laxman Rai
laxmanrai0411@gmail.com

Abstract

Maple is an intent-driven marketplace that allows users to discover, compose, and execute Solana dApps crafted with agentic interfaces rather than fragmented, app-specific user experiences. At its core is a retrieval-augmented generation (RAG) layer that indexes protocol documentation, interface definitions, and on-chain schemas to expose each integrated dApp as a reusable “skill.” With these skills, dApps can be published, updated, and monetized by developers, creating an open marketplace of best-practice workflows for Solana use cases. By combining agentic planning, a RAG-backed knowledge graph, and verifiable on-chain execution integrations, Maple aims to make the Solana ecosystem more accessible to end users while providing a powerful distribution channel for builders and skill creators.

1. Introduction:

Maple is delivered as a progressive web application (PWA), later converted to APK to maximize accessibility and minimize friction across devices. By relying on modern browser capabilities, Maple offers an app-like experience with low storage consumption on user devices, avoiding large native binaries and heavy update cycles. The web ecosystem also provides a richer selection of UI libraries and tooling for both web and web3 compared with mobile-specific stacks, enabling faster iteration, better composability with existing Solana tooling, and a more consistent experience across desktop and mobile browsers.

2. Outline:

Maple’s agent layer consists of specialized agents trained to perform a wide range of on-chain and off-chain skills, from basic wallet operations to complex, multi-protocol workflows. Each agent understands core blockchain functions such as sending transactions, managing approvals, querying on-chain state, and composing interactions across multiple Solana dApps. On top of this foundation, agents integrate with

external tools and services, including wallets, payment rails, and gateways like USDC and Circle, so they can bridge between user intent, on-chain execution, and real-world assets and payments. Once Maple has clarified and validated the user’s intent, the agentic interface transitions from understanding to execution by generating a task-specific UI.

3. Architecture:

A layered system where developers express intent to create production-ready PWAs, which are instantly listed on the Maple marketplace. Users discover dApps through natural language search, preview workflows signature-free, then execute via agentic interfaces. The PWA-first approach leverages rich web3 UI libraries while maintaining minimal storage requirements. Backend services scale horizontally to handle marketplace demand, with pre-built integrations for wallets, payments, RPCs, and Solana protocols ensuring seamless composability.

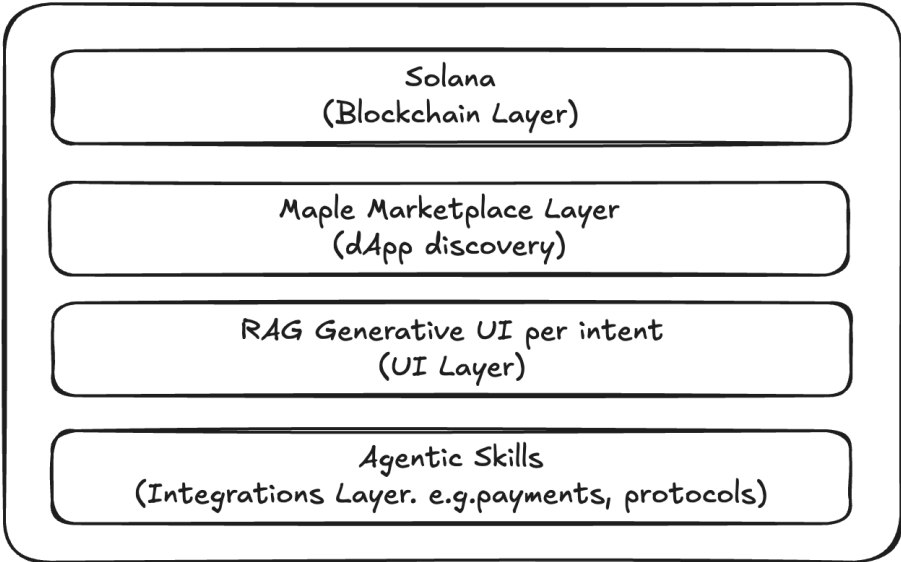


Fig: Maple’s Architecture

4. Platform Selection Model:

We formalize platform selection as an optimization problem over a utility functional $U (P)$ $U(P)$, where $P \in \{ PWA , Mobile , Desktop \}$ $P \in \{PWA,Mobile,Desktop\}$. Each platform is evaluated across development efficiency D , SDK compatibility S , distribution efficiency G , and iteration velocity I , with a bounded penalty for runtime performance R R . The objective is defined as:

$$U(P) = \alpha D + \beta S + \gamma G + \delta I + \epsilon R, \sum \alpha_i = 1$$

$$U(P) = \alpha D + \beta S + \gamma G + \delta I + \epsilon R, \sum \alpha_i = 1$$

In the Web3 context, $S \rightarrow 1$ and $G \rightarrow 1$ dominate due to the browser-native nature of blockchain SDKs and wallet providers, as well as the URL-based distribution model. Progressive Web Apps (PWAs) maximize $S \rightarrow 1$ through direct access to JavaScript SDKs and injected wallet interfaces, while simultaneously minimizing distribution friction $\phi \rightarrow 0$, yielding $G \rightarrow 1$. Additionally, PWAs achieve superior D and I due to a single codebase and the absence of deployment gatekeeping (i.e., $t_{\text{approval}} = 0$).

Although native platforms exhibit marginally higher runtime performance R , this advantage is asymptotically bounded and contributes less to overall utility in typical Web3 workloads, where network latency and protocol interaction dominate execution cost. Therefore, for weightings where $\beta + \gamma + \delta \gg \epsilon$, the solution converges to:

$$P^* = \arg \max U(P) = \text{PWA}$$

This establishes the PWA architecture as the optimal strategy under Web3-aligned utility conditions, maximizing compatibility, reach, and iteration speed while incurring only negligible performance tradeoffs.

5. RAG-based Agentic UI:

“RAG-based Agentic UI” in our system refers to a retrieval-driven interface generation model, where the agent does not rely solely on its internal knowledge to construct UI, but instead queries a structured component registry at generation time. In this case, the registry is built around custom UI components, indexed with metadata such as component names, props, usage patterns, styling variants, and semantic roles (e.g., input, modal, wallet connector). This registry is embedded and made searchable via Retrieval-Augmented Generation (RAG), allowing the agent to dynamically fetch the most relevant UI primitives for a given task.

When a user intent is provided—such as “create a token swap interface”—the agent first converts that intent into a structured query over the registry. The RAG layer retrieves a set of candidate components (e.g., form controls, selectors, buttons, layout containers) along with their schemas and usage examples. Rather than hallucinating UI structures, the

agent grounds its generation in these retrieved artifacts, ensuring that every element it assembles corresponds to a real, validated component. This significantly improves reliability, consistency, and developer alignment, since the generated UI is constrained to what actually exists in the approved design system.

The “agentic” aspect comes from how the system reasons over these retrieved components. It doesn’t simply list them—it composes them into a coherent interface by matching component capabilities to the intent structure, wiring state and interactions, and respecting layout and dependency constraints. Because the components originate from a shared registry (e.g., `shadcn/ui`), the resulting UI maintains visual and behavioral consistency across the application. In effect, RAG transforms UI generation into a grounded assembly process, where the agent acts as a planner and orchestrator, and the registry serves as the source of truth for all possible interface elements.

6. Maple Agent Skills:

The agent skill layer is modeled as an orchestration system over a heterogeneous set of external integrations, where each capability—payments, wallet interaction, market data access, and persistence—is exposed as a modular tool with well-defined interfaces. Rather than embedding these capabilities directly into the agent, the system abstracts them into a unified execution layer, allowing the agent to dynamically invoke the appropriate service based on user intent. For example, USDC payment flows are executed through the Circle API, while wallet interactions are resolved via either mobile-native providers such as Solana Seeker (through its Mobile Wallet Adapter) or standard Solana wallet interfaces. Each integration is treated as a deterministic function that the agent can call, parameterize, and sequence within a larger task.

This orchestration model enables the agent to decompose complex intents into executable workflows. A single user request—such as executing a trade or tracking an asset—may involve fetching real-time market data from external APIs, validating wallet state, initiating a payment or transaction, and persisting results to a database scoped to the user’s context. The agent coordinates these steps by selecting the required tools, passing intermediate outputs between them, and enforcing execution order and constraints. Persistence is handled through a

structured data layer that stores state and preferences aligned with user intent, allowing subsequent interactions to be context-aware and stateful. In effect, the agent operates not as a monolithic system, but as an orchestrator of specialized services, enabling flexible, composable, and extensible Web3 interactions.

7. **Discovery Layer (Marketplace):**

The discovery layer departs from the traditional app store model by treating applications as independently deployable, URL-addressable modules within a PWA environment. Instead of being distributed through centralized marketplaces, applications are exposed as server-side rendered, agentially generated interfaces that can be instantiated on demand. Each application is effectively a self-contained dApp surface—composed at runtime via the agentic UI layer—and delivered directly through the web, eliminating installation friction while still supporting optional “install” semantics native to PWAs. This architecture allows applications to exist as composable endpoints rather than fixed binaries, enabling continuous updates, instant access, and seamless interoperability across contexts.

Crucially, discovery is coupled directly to the blockchain layer, where applications can be created, shared, monetized, or open-sourced without platform intermediation. Developers publish interfaces and logic as modular dApps, which users can fork, extend, or invoke dynamically based on intent. Monetization primitives (e.g., usage fees, subscriptions, or token-gated access) are enforced at the protocol level, rather than through app store policies. This model aligns naturally with mobile-native Web3 environments, particularly devices like Solana Seeker, where deep integration with the Solana ecosystem enables direct wallet interaction, low-latency transactions, and seamless execution of these agent-driven applications. The result is a decentralized discovery layer where applications are not “downloaded” but dynamically resolved, executed, and composed at the edge.

8. **References:**

- *Anthropic: 2026 Agentic Coding Trends Report Discusses the evolution of single agents into coordinated teams for system architecture and dApp building.*
<https://resources.anthropic.com/hubfs/2026%20Agentic%20Coding%20Trends%20Report.pdf>

- **CopilotKit:** *Generative UI Patterns (AG-UI) Documentation for the AG-UI protocol, which bridges agents and frontends to render task-specific React components from a registry.*
<https://github.com/CopilotKit/generative-ui/blob/main/README.md>
- **Solana Mobile Wallet Adapter (MWA)** *Specification The core protocol enabling your PWA to communicate with mobile wallets (like the Solana Seeker) for signature-free previews and verifiable execution.*
<https://docs.solanamobile.com/developers/mobile-wallet-adapter>